Technical Report: City Liveability & Urban Climate Insights API

Module: Web Services and Web Data, University of Leeds

Author: Luke Price

Date: 20 March 2026

Repository: GitHub

Deployed URL: URL

Presentation:: Presentation

---

1. Introduction

This report documents the design, implementation, and evaluation of the City Liveability & Urban Climate Insights API — a data-driven RESTful web service that aggregates urban climate, air quality, and socioeconomic data for UK cities. The system computes composite liveability scores, detects statistical climate anomalies, and delivers city narrative summaries through a structured API. Crucially, it also exposes a Model Context Protocol (MCP) server so that LLM agents such as Claude can interact with the data conversationally via tool calls — a feature that extends the project beyond conventional web API design into the emerging space of AI-accessible data services.

The project was chosen because it sits at the intersection of two personally relevant domains: environmental data and practical AI integration. The open-ended nature of the brief allowed me to build something that demonstrates not just functional CRUD mechanics, but a coherent system with a meaningful analytical purpose.

---

2. Technology Stack Justification

2.1 Framework — FastAPI

FastAPI was chosen over Django REST Framework and Flask for three principal reasons. First, it is natively asynchronous, which matters when performing I/O-bound operations such as fetching weather data from an external API during city creation. Second, it generates interactive OpenAPI documentation (Swagger UI and ReDoc) automatically from Python type annotations, eliminating the need to maintain a separate documentation layer. Third, FastAPI's tight coupling with Pydantic v2 provides compile-time schema validation and serialisation at substantially higher throughput than equivalent Django approaches, which is relevant when stress-testing with the 100 requests-per-minute rate limit.

Django was explicitly evaluated and ruled out: its ORM and request–response cycle carry more overhead than is needed for a pure API service, and its synchronous-by-default model would have required additional configuration to support async database access.

2.2 ORM — SQLAlchemy 2.0

SQLAlchemy 2.0 was selected as the ORM layer for its support of multiple database backends and its mature, well-documented query interface. The decision to separate the ORM from the web framework (rather than using Django's built-in ORM) allows the same model definitions to target both the SQLite

development database and a PostgreSQL production instance without any code changes — only the `DATABASE_URL` environment variable differs.

2.3 Database — SQLite (development) / PostgreSQL (production)

SQLite is used locally because it requires zero configuration and produces a single portable file, accelerating development iteration. The production configuration targets PostgreSQL, hosted via Railway, because it handles concurrent connections safely and provides the full type system required for production data integrity. Alembic is used for schema versioning, ensuring that migrations are tracked in version control and can be applied or rolled back deterministically.

A NoSQL database such as MongoDB was considered for the climate metrics collection, given that time-series readings have a naturally document-like structure. However, SQLite and PostgreSQL were preferred because the relational model allows efficient JOIN queries between cities, climate metrics, and socioeconomic data — which is precisely what the analytics endpoints require — without denormalising data or duplicating city identifiers across nested documents.

2.4 Authentication — JWT with RBAC

Stateless JWT tokens were chosen over session-based authentication because the API is designed to be consumed by both human clients and LLM agents, neither of which benefits from server-side session storage. The `python-jose` library handles token signing with HMAC-SHA256 (HS256), and `bcrypt` is used for password hashing with an appropriate cost factor. Role-based access control (RBAC) distinguishes between `user` and `admin` roles: administrative operations such as deleting cities or climate readings are protected by an admin-only dependency injected at the router level, keeping the authorisation logic centralised rather than duplicated per endpoint.

2.5 External Data — Open-Meteo Archive API

Open-Meteo was chosen as the weather data source because it is free, requires no API key, and provides historical daily climate variables (temperature, precipitation, humidity) through a clean REST interface. When a city is created or updated, a background thread fetches the previous 365 days of climate data and stores it as climate metric rows, automatically skipping dates that are already present to avoid duplication. This approach seeds the database with meaningful real-world data without requiring a separate ETL pipeline.

2.6 MCP Server

The Model Context Protocol server was implemented using Anthropic's `fastmcp` SDK. It exposes nine tools covering city search, liveability computation, anomaly detection, trend retrieval, and observation recording. The MCP server is mounted on the same FastAPI process at `/mcp/sse`, meaning no separate deployment is required. This design enables a Claude Desktop instance to connect to the deployed Railway URL and interact with UK city data conversationally — for example, asking which city has the best air quality trend over the past six months and receiving a computed, data-backed answer.

---

3. Architecture and Design Decisions

The application follows a layered architecture with clear separation of concerns:

- **Routers** handle HTTP routing and request/response serialisation.
- **Services** contain all business logic and are the only layer that touches the ORM session.
- **Models** define the database schema using SQLAlchemy declarative syntax.
- **Schemas** define Pydantic models for request validation and response serialisation.

- **Utils** contain pure functions — the scoring algorithm, security helpers, and the SQL sandbox validator — with no database dependencies, making them straightforward to unit-test in isolation.

This separation means that routers are thin and testable without a live database (by injecting a mock service), while services can be unit-tested by passing an in-memory SQLite session.

3.1 Liveability Scoring

The composite liveability score is computed from four sub-dimensions: climate comfort (25%), affordability (30%), safety (25%), and environment (20%). Each raw metric is normalised to a 0–100 scale using domain-specific functions. For example, temperature comfort is scored by measuring deviation from an ideal UK temperature of 17.5°C, penalising both extreme heat and cold. AQI is inverted — a lower index produces a higher score. Rent and crime index are normalised relative to empirical UK maxima. The weighted sum produces an overall score that is interpretable and comparable across cities.

3.2 Anomaly Detection

Z-score-based anomaly detection was implemented in the analytics service. For a given city and metric, the service computes the population mean and standard deviation across all stored readings, then identifies dates where the absolute z-score exceeds a configurable threshold (defaulting to 2.0). This approach is statistically sound for continuous climate variables and surfaces genuine outliers such as heatwave days or extreme precipitation events without requiring any external ML library.

3.3 Router Ordering

A non-obvious design constraint arose from FastAPI's path matching rules. The `GET /api/v1/cities/compare` endpoint must be registered before `GET /api/v1/cities/{city_id}`, because FastAPI matches routes in registration order and `compare` would otherwise be interpreted as a `city_id` parameter. This is documented explicitly in the application factory (`main.py`) as a comment to prevent future developers from accidentally reordering the router includes.

---

4. Testing Approach

The project includes 228 tests organised into modules that mirror the application structure. Tests are written using `pytest` with `httpx` as the test client, which supports FastAPI's async interface natively.

The test strategy covers several layers:

- **Integration tests** exercise full request–response cycles against an in-memory SQLite database, seeded with known fixtures. These tests verify that CRUD endpoints return correct status codes, response shapes, and pagination envelopes.
- **Unit tests** for services pass a real but temporary SQLite session to verify business logic without the HTTP layer.
- **Security tests** verify that protected routes reject unauthenticated requests with 401, that RBAC enforcement rejects non-admin users on admin-only endpoints with 403, and that JWTs with tampered signatures are rejected.
- **Utility tests** exercise the scoring algorithm and SQL validator with edge-case inputs.
- **Middleware tests** confirm that the rate limiter triggers at the configured threshold and that the logging middleware records latency fields.

A deliberate choice was made to use real SQLite sessions rather than mocked database calls in integration tests, having observed that mocked tests can mask schema-level bugs that only appear when the ORM

generates actual SQL. This trades a small increase in test execution time for substantially higher confidence that the database interaction is correct.

---

5. Challenges and Lessons Learned

Router ordering was the most counterintuitive issue encountered during development. The FastAPI path-matching behaviour described above caused intermittent 422 validation errors on the comparison endpoint until the root cause was identified and the router registration order was fixed.

Background seeding required careful handling to avoid blocking application startup. The Open-Meteo fetch for 365 days of data across 15 cities produces several thousand HTTP requests, so the seed script was moved to a daemon thread that runs after the application is ready to accept requests. A `SKIP_SEED` environment variable was introduced to suppress this behaviour during testing, preventing tests from making real external HTTP calls.

Alembic and SQLite have a well-known incompatibility: SQLite does not support `ALTER COLUMN` operations, which means certain schema changes require table recreation rather than an in-place modification. This was managed by using a batch migration mode in the Alembic environment configuration, which transparently handles the recreation without requiring manual intervention.

---

6. Limitations and Future Development

The current implementation has several known limitations:

Liveability weights are static. The scoring algorithm uses fixed weights (climate 25%, affordability 30%, safety 25%, environment 20%). A future version could expose a `/liveability?weights=...` query parameter allowing callers to specify their own priority profile, or could learn user-specific weights from saved preferences.

Narrative generation is template-based. The city narrative endpoint currently fills a string template with numeric scores. Integrating a language model (via the Claude API) would produce genuinely readable, contextualised narratives that highlight relative strengths and weaknesses compared to peer cities.

No time-based liveability history. The analytics endpoints compute a snapshot score from the most recent data. Tracking how a city's liveability score evolves over time would enable trend endpoints showing whether cities are improving or declining, which would be valuable for policy analysis.

Read replicas and caching. Under production load, repeatedly computing liveability scores from raw climate data is unnecessarily expensive. A caching layer (Redis, or simple in-process `functools.lru_cache` with TTL) on the analytics service would reduce database load significantly.

Frontend portal. A basic HTML frontend is mounted at `/ui`, but it is static. A React or Vue frontend consuming the API would provide a more complete demonstration of the system's capabilities.

---

7. Conclusion

This project demonstrates a complete, production-quality RESTful API built with a modern Python stack. The choice of FastAPI, SQLAlchemy 2.0, and JWT authentication reflects considered trade-offs between development velocity, runtime performance, and maintainability. The addition of an MCP server goes beyond the basic requirements to position the API as an AI-accessible data service — a

design pattern that is increasingly relevant as LLM agents become primary consumers of structured data. The 228-test suite provides confidence that the implementation is correct and that future changes can be validated without regression.

---

8. Generative AI Declaration

Generative AI tools were used at multiple stages of this project. During the ideation phase, AI was used to brainstorm viable project concepts and investigate suitable open datasets for each. Once a direction was chosen, AI assisted in generating an initial code scaffold and boilerplate code — for example, producing model definitions directly from a database diagram — which was then validated against a manually written test suite. AI was also used to review test coverage and suggest additional cases, though debugging failures required manual intervention throughout. Finally, AI assisted in drafting and refining this technical report. Overall, AI proved genuinely useful, particularly in accelerating low-level implementation tasks. This freed up time to focus on higher-level architectural decisions — such as the anomaly detection approach, and MCP integration — rather than spending time on arbitrary boilerplate syntax and conventions.

However, AI assistance had a notable drawback: generated code rarely incorporated proactive refactoring. As the codebase grew, this resulted in accumulating technical debt that required periodic manual intervention to restructure. Future use would benefit from explicitly prompting for refactoring suggestions alongside new code generation.

Appendix Example Gen AI Conversation logs

A.2 Conversation Logs

Log 1 — Ideation Prompt: > "I need to build a web API for a university module. It should use real-world open data, have meaningful analytics, and be something I'm personally interested in. Give me 5 viable project ideas."

Response:

Here are five viable project ideas that combine real-world open data with meaningful analytics:

1. **UK City Liveability API** — Aggregate climate, air quality, crime, and housing affordability data for UK cities to compute a composite liveability score. Analytics could include anomaly detection on climate readings, trend analysis over time, and city-to-city comparisons. Open-Meteo, DEFRA UK-AIR, and ONS datasets are all freely available with no authentication required.

2. **Public Transport Disruption Tracker** — Consume Transport for London and National Rail open feeds to track disruption frequency, average delay times, and worst-performing routes. Analytics could surface which lines are statistically most unreliable and at what times of day.

3. **Green Space Accessibility Scorer** — Using Ordnance Survey open data and ONS population density figures, score UK local authorities by how accessible public parks and green spaces are relative to their population. Could include a deprivation overlay using the Index of Multiple Deprivation.

4. **Flood Risk & Precipitation API** — Combine Environment Agency flood zone data with historical rainfall from Open-Meteo to compute rolling flood risk scores for UK postcodes. Anomaly detection could flag periods of unusually high precipitation against historical baselines.

5. **Regional Energy & Emissions Comparator** — Use DESNZ (Department for Energy Security) open data on regional electricity consumption and carbon intensity to allow users to query and compare energy efficiency trends across UK regions over time.

Of these, the City Liveability API is likely the strongest choice for a university module — it has clearly separable data sources, natural CRUD semantics (cities, readings, users), and the analytics layer (scoring, anomaly detection, comparisons) gives you plenty to discuss in a technical report.

Log 2 — Dataset Investigation **Prompt:** > "For the city liveability API idea, what open datasets are available for UK cities covering climate, air quality, and housing affordability?"

Response:

Here's a breakdown of the most viable open datasets for each dimension of a UK city liveability API:

Climate Data Open-Meteo's Historical Weather API is the strongest option here. It's completely free, requires no API key, and provides daily climate variables including mean/max/min temperature, precipitation sum, windspeed, and relative humidity going back decades. You query it by latitude and longitude, making it straightforward to seed a database with historical readings for any UK city. The API is well-documented and returns clean JSON, so no significant parsing work is required.

Air Quality DEFRA's UK-AIR API provides air quality index readings from monitoring stations across the UK. It's free and publicly accessible, though coverage varies — major cities like London, Manchester, and Leeds have dense monitoring networks, while smaller cities may have only one or two stations. An alternative is the OpenAQ platform, which aggregates air quality data from multiple government sources into a single unified API and may offer better coverage for smaller cities.

Housing Affordability The ONS publishes median private rental prices by local authority on a quarterly basis, available as downloadable CSV files. For house prices, the Land Registry Price Paid dataset is comprehensive and updated monthly, though it requires more preprocessing to derive meaningful per-city affordability metrics. If you want a simpler proxy, the ONS Housing Affordability Ratio (median house price divided by median workplace earnings, by local authority) is pre-computed and straightforward to use.

Crime data.police.uk provides street-level crime data for all UK police forces via a REST API, queryable by latitude, longitude, and date. You could aggregate monthly crime counts per city and normalise by population to produce a crime index.

A note on combining these sources The main challenge will be that these datasets use inconsistent geographic identifiers — Open-Meteo uses coordinates, ONS uses local authority codes, and data.police.uk uses lat/lon bounding boxes. You'll likely want to maintain a city lookup table that stores both coordinates and ONS local authority codes, so you can join across sources cleanly. I'd recommend starting with Open-Meteo and the ONS rental data as your core sources, since both are clean and easy to consume, then layering in crime and air quality once the core schema is established.

Log 3 — Test Coverage Review **Prompt:** > "Here are my existing tests for the /cities endpoints. What cases am I missing?"

Response:

Looking at your existing tests, you have good coverage of the happy path — creating, reading, updating, and deleting cities with valid data. Here are the cases I'd recommend adding:

Pagination edge cases You're not testing boundary conditions on your pagination parameters. Consider adding tests for `page=0` or `page=-1` (should return a 422 validation error), a page value beyond the last available page (should return an empty list, not a 404), and a `page_size` of 0 or an extremely large value. These are common sources of unexpected behaviour in production.

Duplicate city names Your schema likely enforces a unique constraint on city name. You should have a test that attempts to create two cities with the same name and asserts a 409 Conflict response. Without this, the database will raise an integrity error that bubbles up as an unhandled 500.

Filtering with no matching results If your `/cities` endpoint supports query filters (e.g. by country or region), add a test that filters by a value that matches nothing. It should return a 200 with an empty list, not a 404 — a common mistake is to raise a 404 when a query returns zero rows, which is semantically incorrect for a collection endpoint.

RBAC enforcement You have no tests verifying that non-admin users are rejected on admin-only endpoints. Add tests that authenticate as a regular user and attempt a DELETE request, asserting a 403 Forbidden response. Similarly, test that unauthenticated requests to any protected endpoint return 401, not 403.

Tampered and expired JWTs Add a test that sends a request with a JWT whose signature has been modified — this should return a 401. Also test with a token whose `exp` claim is in the past. These verify that your auth middleware is actually validating tokens rather than just decoding them.

Response shape validation Your tests assert status codes but don't always verify the full response shape. For list endpoints, assert that the response envelope contains `items`, `total`, `page`, and `page_size` fields. For individual city responses, assert that all expected fields are present and of the correct type. This guards against accidentally breaking your schema during refactoring.

I'd also suggest running `pytest --cov` and checking which lines in your services and utils aren't covered — there are likely some error-handling branches that are only reachable through edge cases.

Log 4 — Boilerplate Scaffold from DB Diagram **Prompt:** > "Here is my database schema with tables: cities, climate_metrics, socioeconomic_data, users. Generate SQLAlchemy 2.0 models and corresponding Pydantic schemas for each."

Response:

Here are the SQLAlchemy 2.0 models and Pydantic schemas for each of your tables. I've used the declarative `mapped_column` syntax introduced in SQLAlchemy 2.0 rather than the legacy `Column` approach.

SQLAlchemy Models

```
from sqlalchemy import ForeignKey, String, Float, Integer, Date
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship
from datetime import date

class Base(DeclarativeBase):
    pass

class City(Base):
    __tablename__ = "cities"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
    country: Mapped[str] = mapped_column(String(100), nullable=False)
```

```python
    latitude: Mapped[float] = mapped_column(Float, nullable=False)
    longitude: Mapped[float] = mapped_column(Float, nullable=False)
    climate_metrics: Mapped[list["ClimateMetric"]] = relationship(back_populates="city")
    socioeconomic_data: Mapped[list["SocioeconomicData"]] = relationship(back_populates="city")

class ClimateMetric(Base):
    __tablename__ = "climate_metrics"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    city_id: Mapped[int] = mapped_column(ForeignKey("cities.id"), nullable=False)
    date: Mapped[date] = mapped_column(Date, nullable=False)
    temperature: Mapped[float] = mapped_column(Float, nullable=True)
    precipitation: Mapped[float] = mapped_column(Float, nullable=True)
    humidity: Mapped[float] = mapped_column(Float, nullable=True)
    aqi: Mapped[float] = mapped_column(Float, nullable=True)
    city: Mapped["City"] = relationship(back_populates="climate_metrics")

class SocioeconomicData(Base):
    __tablename__ = "socioeconomic_data"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    city_id: Mapped[int] = mapped_column(ForeignKey("cities.id"), nullable=False)
    median_rent: Mapped[float] = mapped_column(Float, nullable=True)
    crime_index: Mapped[float] = mapped_column(Float, nullable=True)
    city: Mapped["City"] = relationship(back_populates="socioeconomic_data")

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(20), nullable=False, default="user")
```

Pydantic Schemas

```python
from pydantic import BaseModel, EmailStr
from datetime import date
from typing import Optional

class CityBase(BaseModel):
    name: str
    country: str
    latitude: float
    longitude: float

class CityCreate(CityBase):
    pass

class CityResponse(CityBase):
    id: int
    model_config = {"from_attributes": True}

class ClimateMetricBase(BaseModel):
    date: date
    temperature: Optional[float] = None
```

```python
    precipitation: Optional[float] = None
    humidity: Optional[float] = None
    aqi: Optional[float] = None

class ClimateMetricCreate(ClimateMetricBase):
    city_id: int

class ClimateMetricResponse(ClimateMetricBase):
    id: int
    city_id: int
    model_config = {"from_attributes": True}

class SocioeconomicDataBase(BaseModel):
    median_rent: Optional[float] = None
    crime_index: Optional[float] = None

class SocioeconomicDataCreate(SocioeconomicDataBase):
    city_id: int

class SocioeconomicDataResponse(SocioeconomicDataBase):
    id: int
    city_id: int
    model_config = {"from_attributes": True}

class UserCreate(BaseModel):
    email: EmailStr
    password: str

class UserResponse(BaseModel):
    id: int
    email: str
    role: str
    model_config = {"from_attributes": True}
```

Note that I've kept `UserCreate` separate from `UserResponse` and excluded `hashed_password` from the response schema — you should never return password hashes in API responses. You'll want to add a service layer function that takes a `UserCreate`, hashes the password with bcrypt, and persists a `User` model instance.

Log 5 — Technical Debt Example **Prompt:** > "Add anomaly detection to my analytics service. It should compute z-scores for a given city and metric and return dates where the z-score exceeds a configurable threshold."

Response:

Here's a function you can add directly to your `analytics_service.py`:

```python
def detect_anomalies(
    db: Session,
    city_id: int,
    metric: str,
    threshold: float = 2.0
```

```python
) -> list[dict]:
    allowed_metrics = {"temperature", "precipitation", "humidity", "aqi"}
    if metric not in allowed_metrics:
        raise ValueError(f"Invalid metric: {metric}")

    readings = (
        db.query(ClimateMetric)
        .filter(ClimateMetric.city_id == city_id)
        .all()
    )

    values = [getattr(r, metric) for r in readings if getattr(r, metric) is not None]
    if len(values) < 2:
        return []

    mean = sum(values) / len(values)
    std = (sum((v - mean) ** 2 for v in values) / len(values)) ** 0.5
    if std == 0:
        return []

    anomalies = []
    for reading in readings:
        value = getattr(reading, metric)
        if value is not None:
            z = (value - mean) / std
            if abs(z) > threshold:
                anomalies.append({
                    "date": reading.date,
                    "value": value,
                    "z_score": round(z, 3)
                })

    return sorted(anomalies, key=lambda x: abs(x["z_score"]), reverse=True)
```

Add a corresponding route to your analytics router:

```python
@router.get("/cities/{city_id}/anomalies")
def get_anomalies(
    city_id: int,
    metric: str = Query(...),
    threshold: float = Query(default=2.0),
    db: Session = Depends(get_db)
):
    return detect_anomalies(db, city_id, metric, threshold)
```

This should integrate cleanly with your existing service structure.

(Note: the mean and standard deviation computation was written inline rather than extracted into a shared `compute_z_scores` utility — when the same statistical logic was later needed for the trend analysis endpoint, this required manual refactoring to avoid duplication.)

Log 6 — Report Writing Prompt: > "Write a technical justification for choosing FastAPI over Django REST Framework and Flask for a university web services project. The project is a data analytics API with JWT authentication and an MCP server."

Response:

Here is a technical justification you can adapt for your report:

FastAPI was selected as the web framework in preference to both Django REST Framework and Flask for three principal reasons.

First, FastAPI is natively asynchronous, built on Starlette and designed around Python's `async/await` model. For a data analytics API that performs I/O-bound operations — fetching historical climate data from an external provider during city creation, and serving multiple concurrent requests during load testing — this is a meaningful advantage. Django's request–response cycle is synchronous by default, and while async views are supported from Django 3.1 onwards, the ORM and many third-party packages still lack full async support, requiring additional configuration. Flask similarly defaults to synchronous execution and requires an extension such as Quart to support async handlers properly.

Second, FastAPI generates interactive OpenAPI documentation automatically from Python type annotations and Pydantic schemas, producing both Swagger UI and ReDoc interfaces without any additional configuration. For a university project where the API must be demonstrable and self-documenting, this eliminates an entire documentation maintenance burden. Django REST Framework provides its own browsable API but requires explicit schema configuration for OpenAPI output; Flask has no built-in documentation generation.

Third, FastAPI's tight integration with Pydantic v2 provides request validation, response serialisation, and schema generation from a single source of truth. This is particularly relevant for the MCP server component, where tool input schemas are derived directly from Pydantic models — ensuring that the schemas exposed to LLM agents are always consistent with the underlying API validation logic.

Django REST Framework was explicitly evaluated and ruled out: its ORM and middleware stack carry overhead that is unnecessary for a pure API service, and its synchronous-by-default model would have required additional configuration to support the async database access patterns used in this project.